

Project Darpan: Unified Business Identity & Activity Graph

This document outlines the proposed architecture and solution for the Karnataka Commerce & Industries hackathon, addressing the challenge of isolated business data and activity tracking without modifying source systems.

1. Problem Understanding & Real-World Constraints

The core challenge is that Karnataka's business data exists in isolated, siloed operational systems. Because these systems were built independently, they lack a unified schema and consistent primary keys. Implementing a "big-bang" master data management (MDM) migration is politically and technically impossible.

The solution requires an **overlay architecture**—a non-intrusive layer that observes data from existing systems and stitches it together probabilistically. The most critical constraint is that **a wrong merge is worse than a missed merge**. Therefore, the system must treat source records as immutable facts and treat linkages as "beliefs" with varying degrees of confidence. These beliefs must be fully explainable, easily reversible, and compliant with data privacy rules.

2. Part A: Entity Resolution & UBID Assignment

To handle varying formats, missing PAN/GSTIN, and legacy data quality, the system will use a multi-stage Entity Resolution Pipeline.

- **Step 1: Ingestion & Standardization:** Ingest read-only data dumps. Normalize strings (e.g., "Pvt Ltd" vs "Private Limited"), standardize addresses using a localized Indian address parser, and clean PAN/GSTIN regex patterns.
- **Step 2: Blocking:** Group potential matches using combinations like [Pin Code + Soundex(Company Name)] to reduce computational load.
- **Step 3: Pairwise Scoring Engine:** Within each block, a deterministic and probabilistic scoring engine evaluates record pairs.
- **Step 4: Graph-Based Clustering (The UBID):** Records are treated as nodes in a Graph Database. If the pair score exceeds the auto-link threshold, a "Same-As" edge is created. A UBID is simply the ID assigned to a clustered connected component in this graph.

3. Confidence Calibration & Thresholds

Every "Same-As" edge will store its calculation features. A lightweight ML model (like XGBoost) outputs a probability score from 0.0 to 1.0.

Score Range	Action	Description
> 0.90	Auto-Link	Near-perfect match; edge drawn automatically.
0.70 – 0.89	Pending Review	Ambiguous match; sent to human reviewer.
< 0.70	Reject	Automatically discarded; records remain separate.

4. Human-in-the-Loop & Active Learning

Ambiguous matches are routed to a Reviewer Dashboard. The UI presents a side-by-side "diff" view highlighting discrepancies. Reviewer decisions (Merge/Separate) feed into a labeled training dataset, allowing the system to continuously learn local data quirks via weekly batch retraining.

5. Part B: Activity Inference

Activity status is derived from a one-way stream of events (renewals, inspections) attached to source records and rolled up to the UBID.

- **Active:** Verified events within the last X months.
- **Dormant:** No events for > 18 months, or failed communications.
- **Closed:** Explicit closure/surrender event received.

6. Respecting Non-Negotiables

No Source System Changes: Utilizes Change Data Capture (CDC) or API batch reads.

No Hosted LLMs on PII: Models run locally within the secure state environment.

Reversibility: Reversing a merge is an O(1) operation of deleting a graph edge.

7. Architecture Overview

1. **Ingestion Layer:** Apache Kafka / Debezium.
2. **Processing Layer:** Apache Spark / Python for bulk data cleaning.
3. **Matching Engine:** Python rules engine + XGBoost.
4. **Storage:** GraphDB (Neo4j) for linkages + Elasticsearch for natural language querying.
5. **Frontend API:** GraphQL or REST API for dashboards.